

第9章 ポインタ (C言語)

S.Matoike : 16 : 00~、19/Jan/2024

第9章で学習すること

キーワードは「ポインタ」

- 課題1 : (int a;) メモリの中での変数 a の領域
- 課題2 : (int *p;) メモリの中でのポインタ p の領域
- 課題3 : 関数の引数に、変数の先頭アドレスの値を渡す呼び出し方
- 課題4 : (応用問題) 統計処理のためのプログラム
- 課題5 : (int **p;) ポインタに関する頭の体操

課題1：メモリの中のアドレス

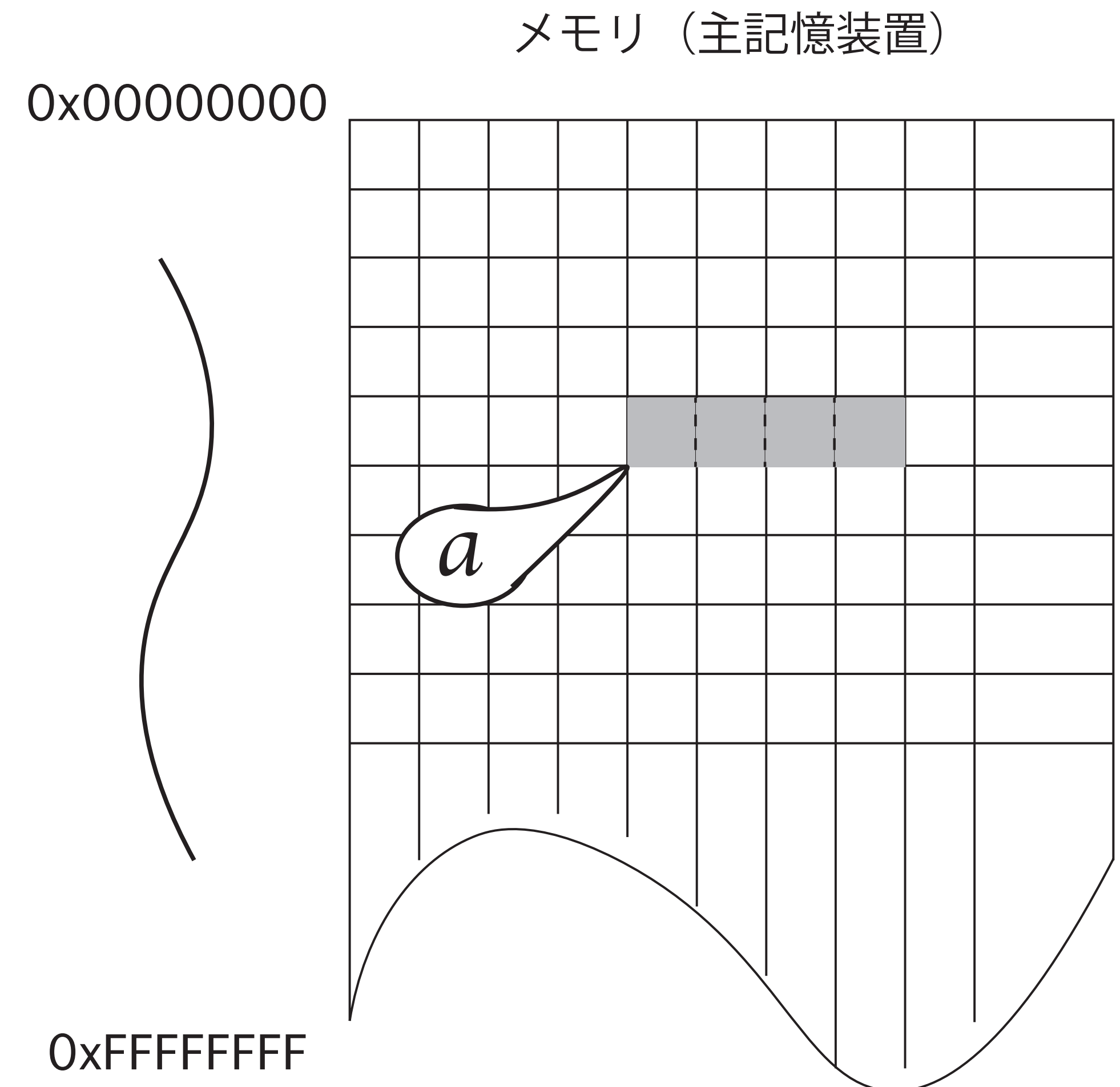
Sample09_01

```
#include <stdio.h>
void main(){
    int a = 10;
    int b;
    printf("a の内容：\t\t%08X(%d)\n", a, a);
    printf("b の内容：\t\t%08X(%d)\n", b, b);
    printf("a の先頭アドレス(&a)：\t%08X\n", &a);
    printf("b の先頭アドレス(&b)：\t%08X\n", &b);
    printf("a のサイズ：\t\t%d バイト\n", sizeof(a));
}
```


int a ;

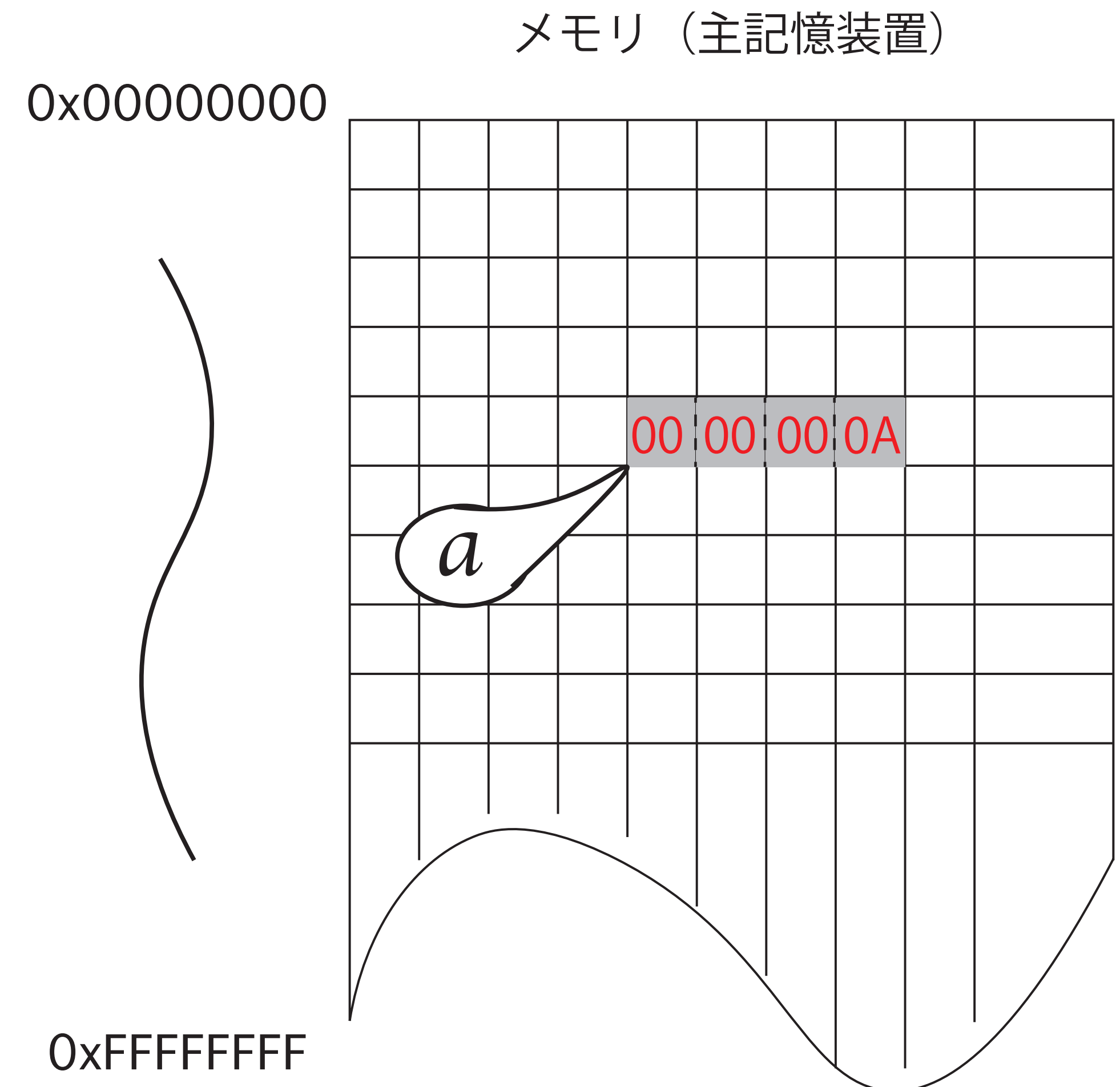
コンパイラがしていること

1. メモリ（主記憶装置）内に、
int(整数)型のデータを収める場所
を確保する
2. その領域に、a というラベルを付
ける
3. その領域の値は「未定」である
(何が入っているか分からない)



int a = 10; コンパイラがしていること

1. メモリ（主記憶装置）内に、
int(整数)型のデータを収める場所
を確保する
2. その領域に、a というラベルを付
ける
3. その領域に、10
(0x0000000A) を代入する



課題1の実行結果

(アドレスは下4バイトのみ)

```
a の内容：          00000000A(10)
b の内容：          0A301BD0(170925008)
a の先頭アドレス(&a)： B612A7DC
b の先頭アドレス(&b)： B612A7D8
a のサイズ：        4 バイト
```


課題2：ポインタ

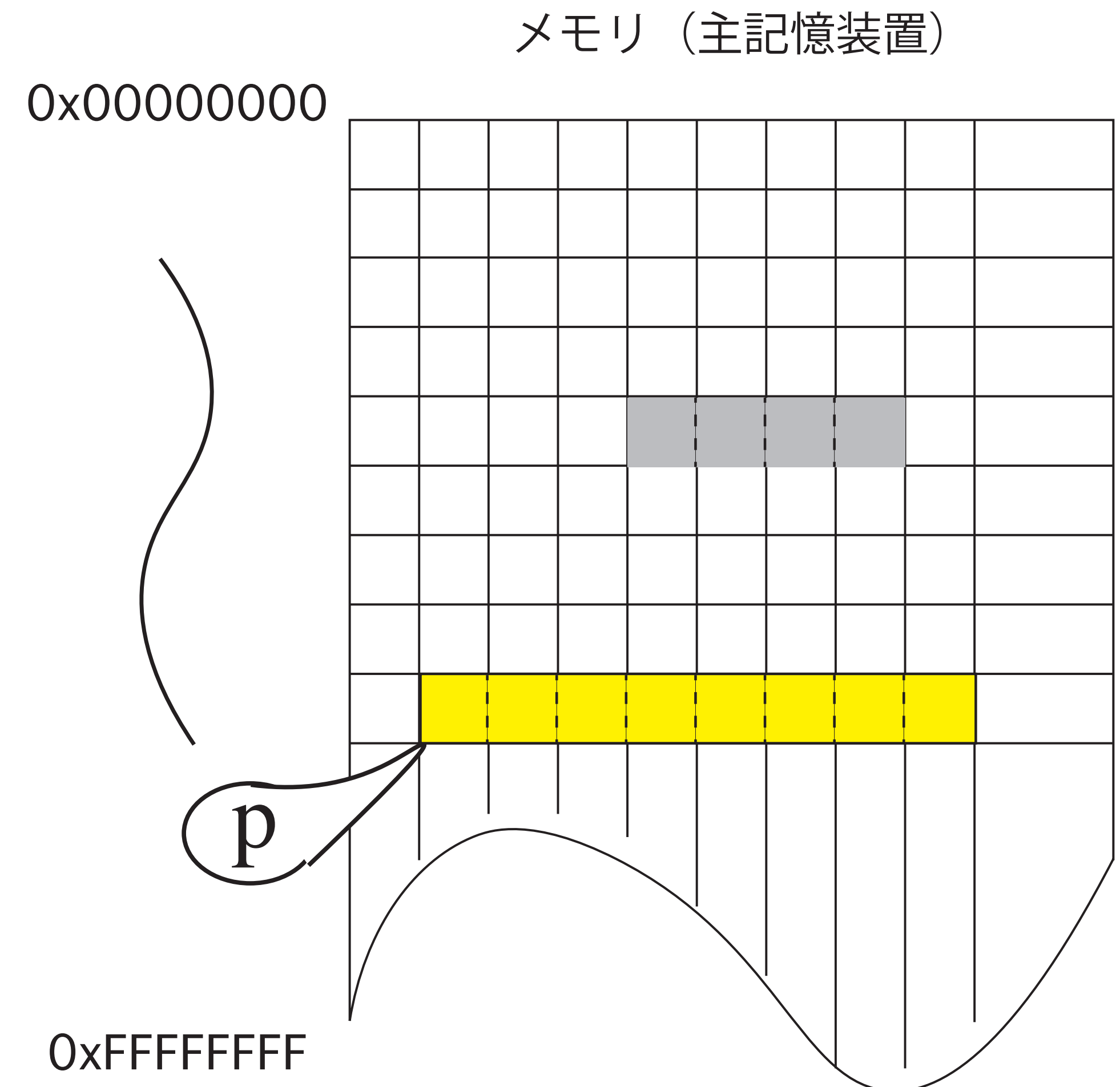
Sample09_02

```
#include <stdio.h>
void main(){
    int a = 10;
    int *p;
    printf("a の内容：\t\t\t%d\n", a);
    printf("a の先頭アドレス(&a)：\t\t\t%08X\n", &a);
    printf("p の内容(場所を確保しただけ)：\t\t\t%08X\n", p);
    p = &a;
    printf("p の内容(aの所在地 &a をpに代入後)：\t\t\t%08X\n", p);
    printf("p が指し示す場所にあるもの(*p)：\t\t\t%d\n", *p);
    a++;
    printf("変数 a の内容をインクリメントした後の *p：\t\t\t%d\n", *p);
    printf("もう一回インクリメントした(++*p)：\t\t\t%d\n", ++*p);
    printf("p のサイズ：\t\t\t\t%d バイト\n", sizeof(p));
}
```

int *p ;

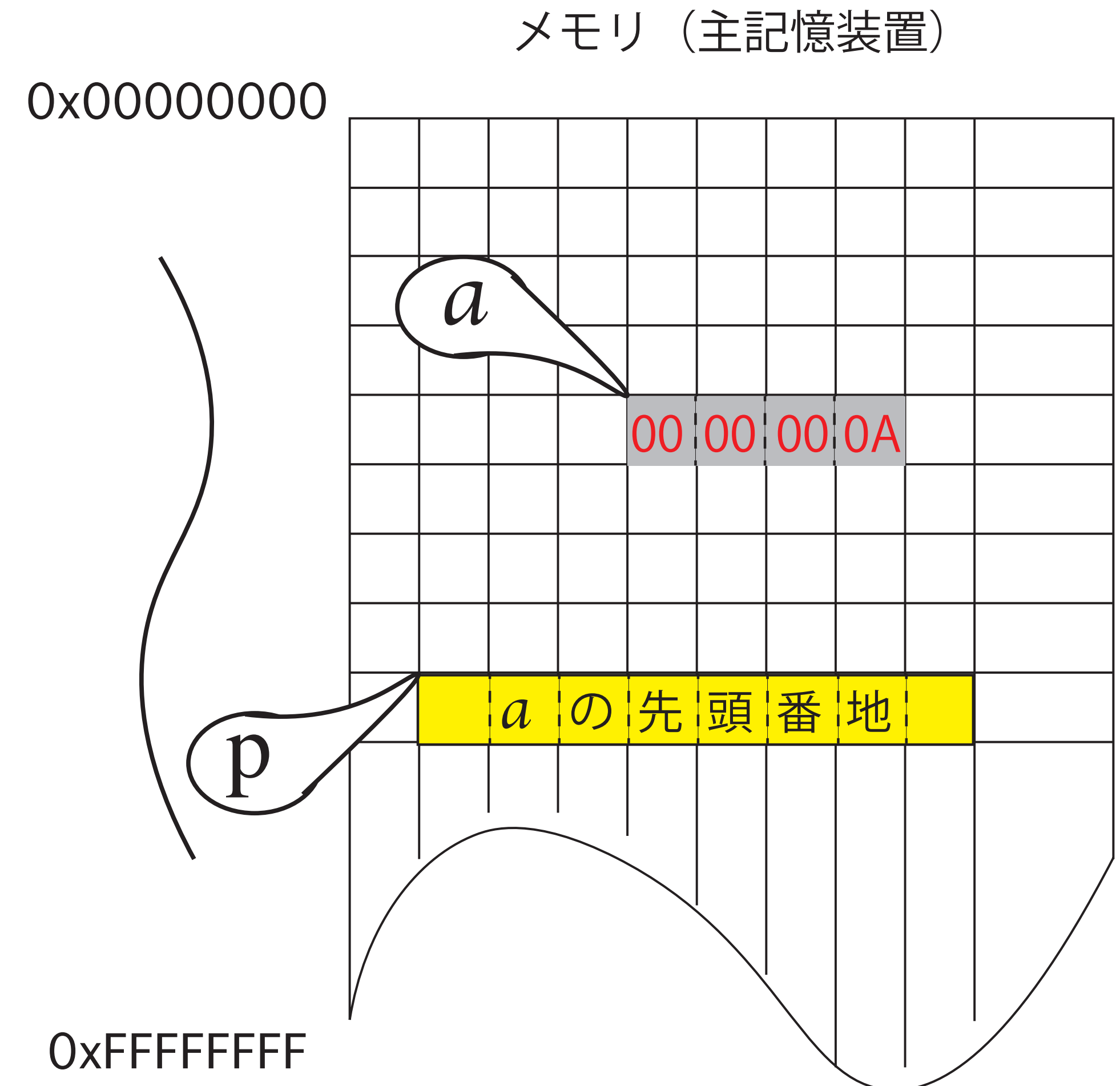
「ポインタが指し示す場所」にあるものは整数

1. メモリ内に、「メモリ上のアドレス」を収めるための場所（ポインタという）を確保する
2. その領域にp というラベルを付ける
3. この段階で、pの内容は未定である（何が入っているか分からない）
4. ポインタの前に*を施す（*p）と、pが指し示す場所にあるものという意味になる
5. ポインタ p が指し示す場所にあるもの（*p）は、int、整数型データである



$p = \&a;$; コンパイラがしていること

1. 変数 a の前に $\&$ を施すと、「 a の所在地の先頭アドレス」という意味になる
2. ポインタ p に「 a が所在する場所の先頭アドレス」を代入している（この段階で p の内容は確定する）
3. 変数 a は `int`、整数型データ でした
4. $*p$ （ポインタ p が指し示すデータ）も `int`、整数型データ でしたね



課題2の実行結果

(アドレスは下4バイトのみ)

a の内容 :	10
a の先頭アドレス(&a) :	B09977DC
p の内容(場所を確保しただけ) :	B0997810
p の内容(aの所在地 &a をpに代入後) :	B09977DC
p が指し示す場所にあるもの(*p) :	10
変数aの内容をインクリメントした後の*p :	11
もう一回インクリメントした(++*p) :	12
p のサイズ :	8 バイト

課題3：関数への引数は値渡し(call by value)

Sample09_03

```
#include<stdio.h>

void swap(int x, int y){
    int temp;
    temp = x;
    x = y;
    y = temp;
}

void main(){
    int a = 10;
    int b = 20;
    printf("前：a=%d\tb=%d\n", a,b);
    swap(a, b);
    printf("後：a=%d\tb=%d\n", a,b);
}
```

```
#include<stdio.h>

void swap(int a, int b){
    int temp;
    temp = a;
    a = b;
    b = temp;
}

void main(){
    int a = 10;
    int b = 20;
    printf("前：a=%d\tb=%d\n", a,b);
    swap(a, b);
    printf("後：a=%d\tb=%d\n", a,b);
}
```

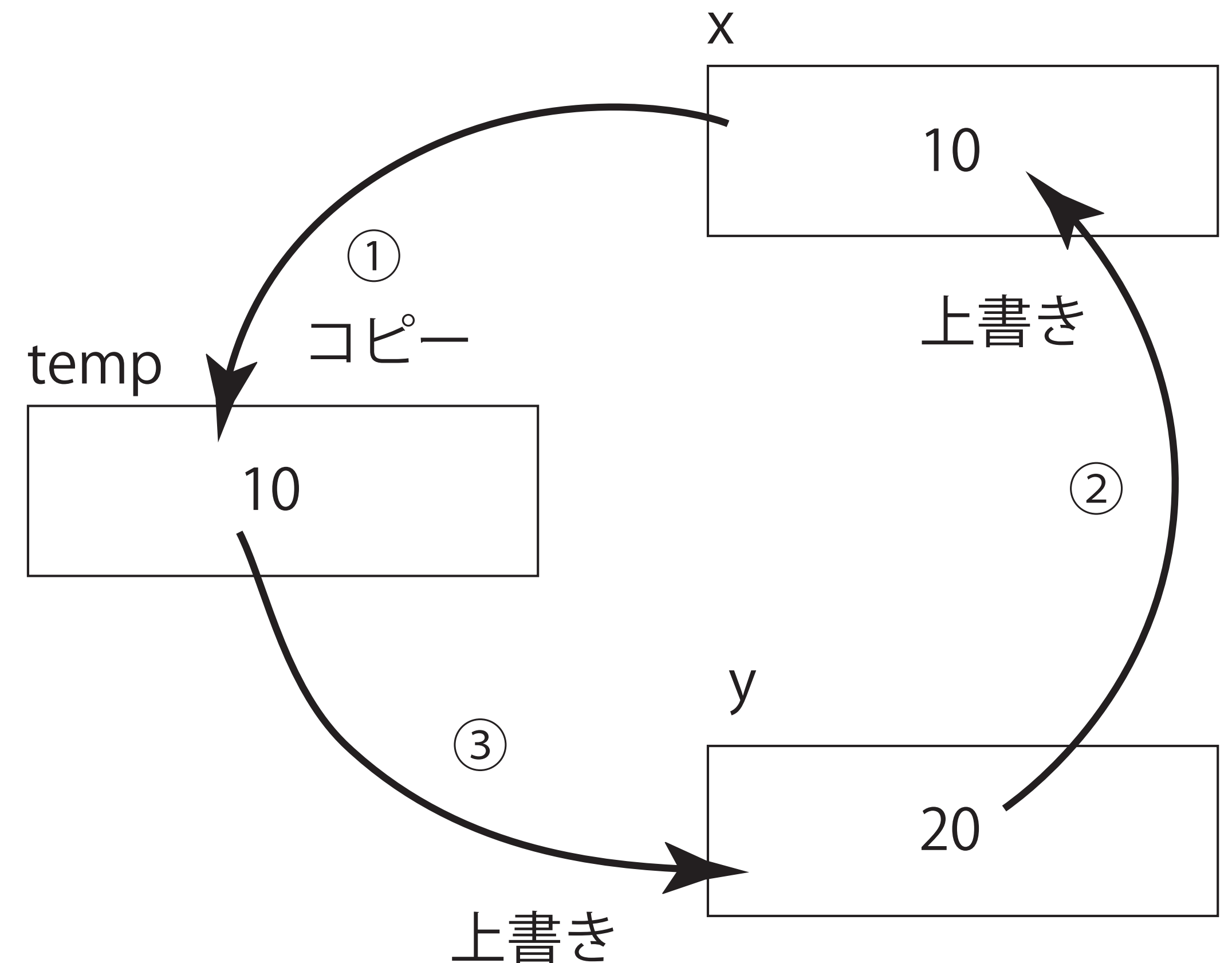

変数の値を交換する関数

(aとbは実引数、xとyは仮引数)

```
void swap(int x, int y){  
    int temp;  
    temp = x;    /* ① */  
    x = y;       /* ② */  
    y = temp;    /* ③ */  
}
```

※ 一見うまくいきそうなのだが、実は . . .

```
int a=10, b=20;  
swap(a, b);
```



こうすると期待通りの動作になる

```
#include<stdio.h>

void swap(int *x, int *y){
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}

void main(){
    int a = 10;
    int b = 20;
    printf("前：a=%d\tb=%d\n", a,b);
    swap(&a, &b);
    printf("後：a=%d\tb=%d\n", a,b);
}
```

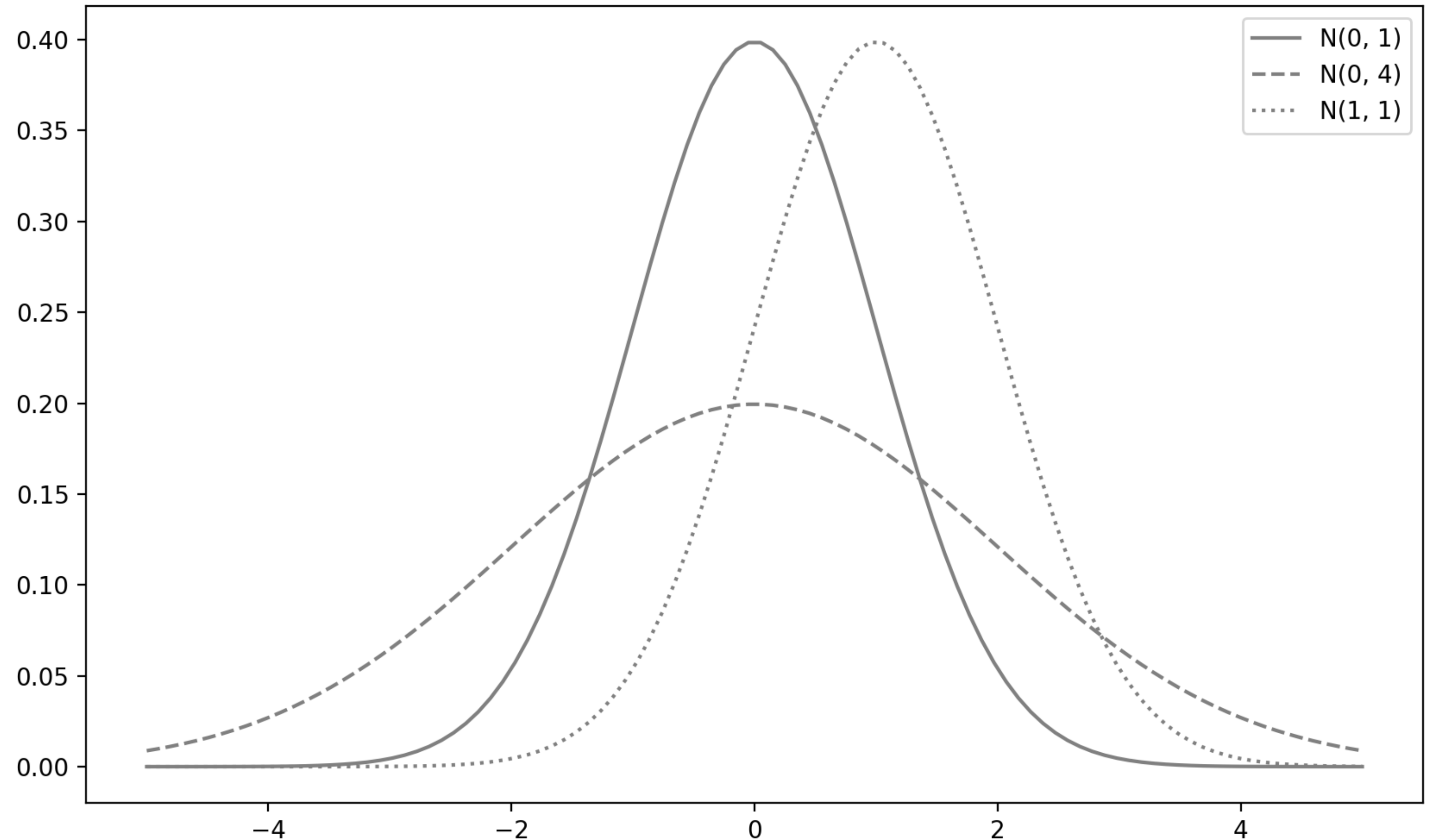
- 関数swap()への実引数 (&aの値=a のアドレス) が、仮引数 (x) に渡される
- 関数swap()への実引数 (&bの値=b のアドレス) が、仮引数 (y) に渡される

前：	a=10	b=20
後：	a=20	b=10

課題4：偏差値と正規分布

章末問題

- $N(\mu, \sigma)$
- $\mu \pm 1\sigma : 68.28\%$
- $\mu \pm 2\sigma : 95.44\%$
- $\mu \pm 3\sigma : 99.73\%$



主処理

関数の引数に配列を渡す

- ① データ入力 : dinput()
- ② 合計、平均 : sum(), mean
- ③ 偏差 : deviation()
- ④ 分散、標準偏差 : variance(), sdev
- ⑤ 標準化 : standardize()
- ⑥ 偏差値 : 平均 + 標準偏差 * Zscore
- ⑦ 結果出力 : doutput()

```
#include <stdio.h>
#include <math.h>
#define NINZ (5)
void main(){
    int tensu[NINZ];
    float mean,var,sdev,hensa[NINZ],zscore[NINZ];
    dinput(tensu);
    mean = (float)sum(tensu) / NINZ;
    deviation(hensa, tensu, mean);
    var = variance(hensa);
    sdev = sqrt(var);
    printf("\n平均 : %f\t標準偏差 : %f\n\n",mean,sdev);
    standardize(zscore, hensa, sdev);
    doutput(tensu, 50, 10, zscore);
}
```

データの入力

入力データの範囲は、0点～100点

範囲外のデータが入力されたら、再入力を促す

```
void dinput(int tensu[]){  
    for(int i=0; I<NINZ; i++){  
        do{  
            printf("点数を入力して (%2d人目) :", i+1);  
            scanf("%d", &tensu[i]);  
        }while(tensu[i]<0||100<tensu[i]);  
    }  
}
```

```
dinput(tensu);
```


合計と平均

合計： $\sum_{i=1}^n x_i = x_1 + x_2 + \cdots + x_n$

平均： $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \frac{1}{n} (x_1 + x_2 + \cdots + x_n)$

```
int sum(int tensu[]){  
    int s = 0;  
    for(int i=0; I<NINZ; i++){  
        s += tensu[i];  
    }  
    return s;  
}
```

```
mean = (float)sum(tensu) / NINZ;
```

偏差

各データの平均からの差（足し合わせると0になる）

$$\text{偏差} : x_i - \bar{x}$$

```
void deviation(float hensa[],int tensu[],float mean){  
    for(int i=0; I<NINZ; i++){  
        hensa[i] = (float)tensu[i] - mean;  
    }  
}
```

```
deviation(hensa, tensu, mean);
```

分散、標準偏差

$$\text{分散} : \sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 = \frac{1}{n} \{ (x_1 - \bar{x})^2 + (x_2 - \bar{x})^2 + \cdots + (x_n - \bar{x})^2 \}$$

$$\text{標準偏差} : \sigma = \sqrt{\sigma^2}$$

```
float variance(float hensa[]){  
    float s2=0;  
    for(int i=0; i<NINZ; i++){  
        s2 += hensa[i]*hensa[i];  
    }  
    return s2 / NINZ;  
}
```

```
var = variance(hensa);  
sdev = sqrt(var);
```


標準化：Zscore（平均0、標準偏差1）

$$\text{基準化変量} : z_i = \frac{x_i - \bar{x}}{\sigma}$$

```
void standardize(float zscore[], float hensa[], float sdev){  
    for(int i=0; I<NINZ; i++){  
        zscore[i] = hensa[i] / sdev;  
    }  
}
```

```
standardize(zscore, hensa, sdev);
```

偏差値＝平均＋標準偏差＊Zscore

$$\text{偏差値} : 50 + 10 \times z_i = 50 + 10 \times \frac{x_i - \bar{x}}{\sigma}$$

```
void doutput(int tensu[], float mean, float variance, float zscore[]){  
    for(int i=0; I<NINZ; i++){  
        float v = mean + variance * zscore[i];  
        printf("%2d人目の点数 : %3d\t偏差値 : %6.2f\n", i+1, tensu[i], v);  
    }  
}
```

```
doutput(tensu, 50, 10, zscore);
```

点数を入力して（1人目）：10

点数を入力して（2人目）：20

点数を入力して（3人目）：30

点数を入力して（4人目）：40

点数を入力して（5人目）：50

平均：30.00 標準偏差：14.14（分散：200.00）

1人目の点数：10 偏差値：35.86

2人目の点数：20 偏差値：42.93

3人目の点数：30 偏差値：50.00

4人目の点数：40 偏差値：57.07

5人目の点数：50 偏差値：64.14

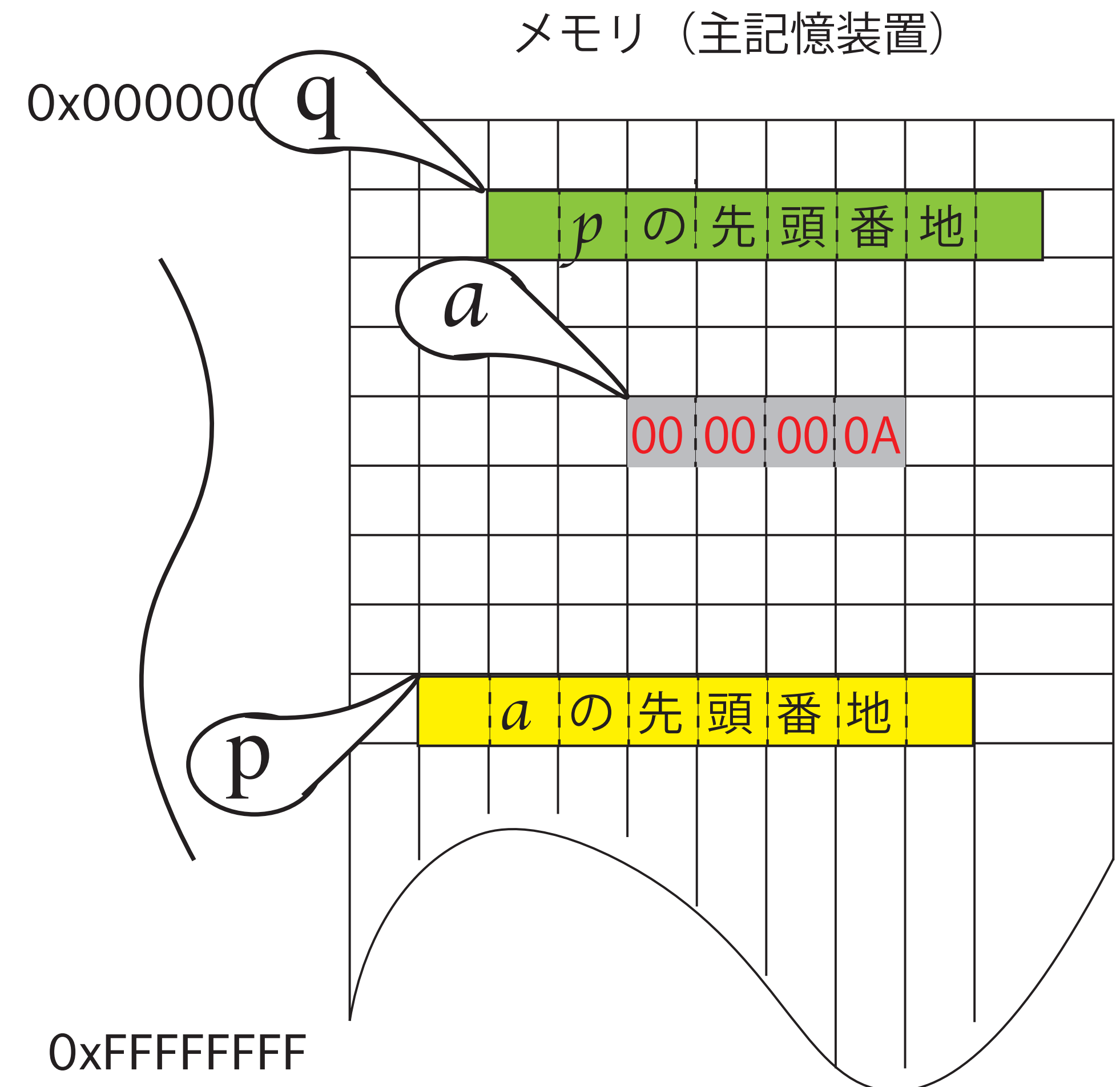
課題5：ポインタのポインタ（頭の体操）

`++**&p` は右から左に向かって順に評価していく

```
#include<stdio.h>
void main(){
    int a = 10;
    int *p, **q;
    printf("a の内容：%d\n", a);
    printf("a の先頭アドレス(&a)：%08X\n", &a);
    p = &a;                                /* p は a へのポインタ */
    printf("p=&a によって *p は：%d\n", *p);
    q = &p;                                /* q は p へのポインタ */
    printf("q=&p によって **q は：%d\n", **q);
    printf("++**q は：%d\n", ++**q);        /* *q は a へのポインタ */
    printf("更に、++**&p は：%d\n", ++**&p); /* *&p は a へのポインタ */
}
```

int *p, **q; コンパイラがしていること

1. メモリ内に、「メモリ上のアドレス」を収めるための場所 p と q を確保している
(この段階で、値は確定していない)
2. *p (ポインタ p が指し示すデータ) は int である
3. **q (ポインタ *q が指し示すデータ) は int である
4. *q (ポインタ q が指し示すデータ) は ポインタである



課題5の実行結果

(アドレスは下4バイトのみ)

a の内容 : 10

a の先頭アドレス(&a) : B145D7DC

p=&a によって *p は : 10

q=&p によって **q は : 10

++**q は : 11

更に、++**&p は : 12